Lecture 2
注意力机制与 Transformer 架构

孔树锋
中山大学　软件工程学院
人工智能与大模型（III）（理论课）



长上下文的计算与存储

输入序列：一次处理整个提示，还是逐个生成 token?

历史信息：逐位置保存，还是汇入一个状态?

上下文访问：读取全部历史，还是选择部分区域?

硬件执行：计算量、显存容量与数据搬运

本节内容

CONTENTS

Self-Attention 与 Transformer

Multi-head Latent Attention（MLA）

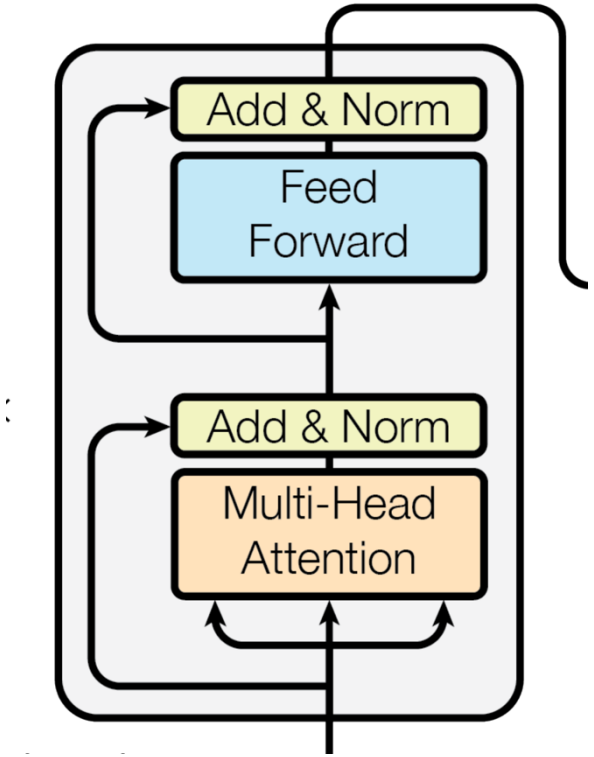
Sparse Attention 与硬件执行

Gated DeltaNet 与 Hybrid Attention

Video Diffusion Transformer

从注意力计算到完整的 Transformer Block

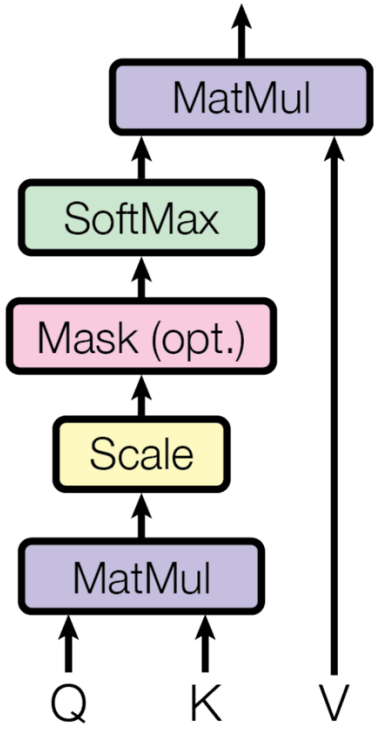
Transformer Block：Residual 与 Norm



Feed-Forward Network（FFN）：同一网络分别处理各位置

Residual Connection（残差连接）：子层输出与原输入向量相加

Scaled Dot-Product Attention



Query（查询）与 Key（键）匹配，得到注意力权重
Value（值）提供内容：按权重相加，得到新表示

缩放点积注意力

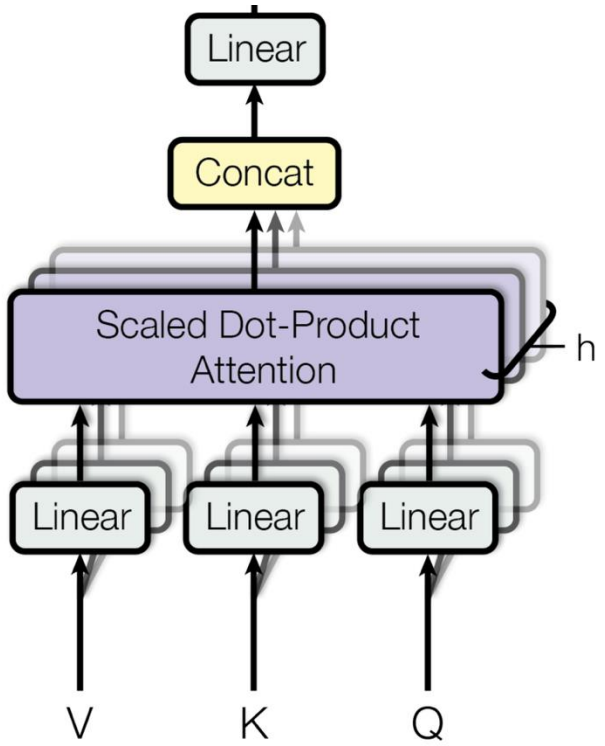
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

QK^T：位置间打分；d_k：Query / Key 的特征维度
Softmax：每行指数化后除以总和，再加权汇总 V

Causal Mask：可见位置

当前查询	允许访问	禁止访问
位置 A	A	B、C
位置 B	A、B	C
位置 C	A、B、C	无

Multi-Head Attention（MHA）



Head（头）：各用一组矩阵生成 Q、K、V，再分别算注意力
Concat 拼接各头输出；Linear 用矩阵把结果变回模型维度

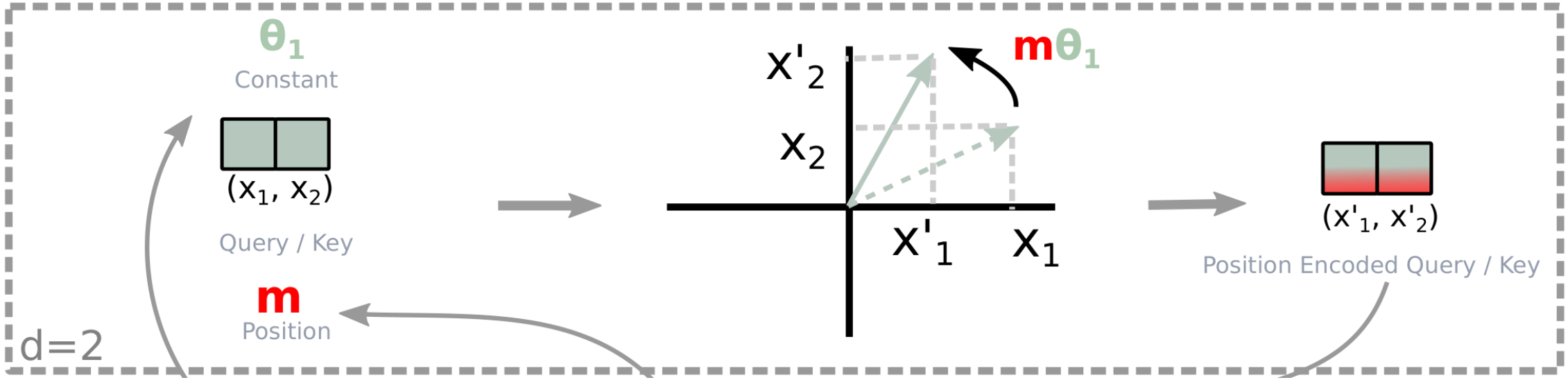
Positional Encoding

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

pos 是位置；i 区分频率；sin / cos 生成位置向量
位置向量与 token embedding（文本单元的向量表示）相加

Rotary Position Embedding（RoPE)



单个特征对 (x_1, x_2)：位置 m 决定旋转角 $m\theta_1$
对 Query 和 Key 分别做上述旋转，得到新的特征对

RoPE：旋转矩阵

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W^{(11)}_{\{q,k\}} & W^{(12)}_{\{q,k\}} \\ W^{(21)}_{\{q,k\}} & W^{(22)}_{\{q,k\}} \end{pmatrix} \begin{pmatrix} x^{(1)}_m \\ x^{(2)}_m \end{pmatrix}$$

cos / sin 矩阵旋转 Query、Key；m 是位置，θ 是频率
共同平移两位置，旋转角度差不变，点积保留相对位置

LayerNorm 与 RMSNorm

LayerNorm（Layer Normalization）

$$\bar{a}_i = \frac{a_i - \mu}{\sigma} g_i, \qquad \mu = \frac{1}{n}\sum_{i=1}^{n} a_i, \quad \sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(a_i - \mu)^2}.$$

RMSNorm（Root Mean Square Normalization）

$$\bar{a}_i = \frac{a_i}{\mathrm{RMS}(\mathbf{a})} g_i, \quad \text{where } \mathrm{RMS}(\mathbf{a}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2}.$$

LayerNorm：减均值再缩放；RMSNorm：不减均值
原式省略 ε；g_i 为可学习缩放，偏置可另加

Position-wise Feed-Forward Network

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Rectified Linear Unit（ReLU）：max(0, x)

同一组参数逐位置执行：线性变换 → 非线性 → 线性变换

Encoder–Decoder 与 Decoder-only

结构	输入与生成	典型注意力连接
Encoder–Decoder	分别编码输入、生成输出	Self-Attention + Cross-Attention
Decoder-only	提示与回答组成自回归序列	Causal Self-Attention

本节内容

CONTENTS

Multi-head Latent Attention（MLA）

多头潜在注意力：以低维表示压缩历史内容

KV Cache = Key–Value Cache（键值缓存）

目的：减少逐 token 保存 Key、Value 的空间

从逐 token 的历史缓存到低秩表示

语言模型推理：Prefill 与 Decode

自回归：根据已有内容预测下一个 token，再接回序列

生成阶段	怎样工作	为何这样计算
Prefill 预填充	处理整段提示，建立 KV 缓存 预测首个回答 token	提示已经给出 各位置可并行计算
Decode 逐步生成	输入刚生成的 token 读取历史 KV、追加当前 KV 预测下一个 token	后续内容尚未知 依赖此前生成结果

缓存原理：因果注意力下，旧位置的 Key、Value 不变

MLA 的作用：压缩历史表示，减少键值缓存占用

KV Cache：键值缓存的保存与读取

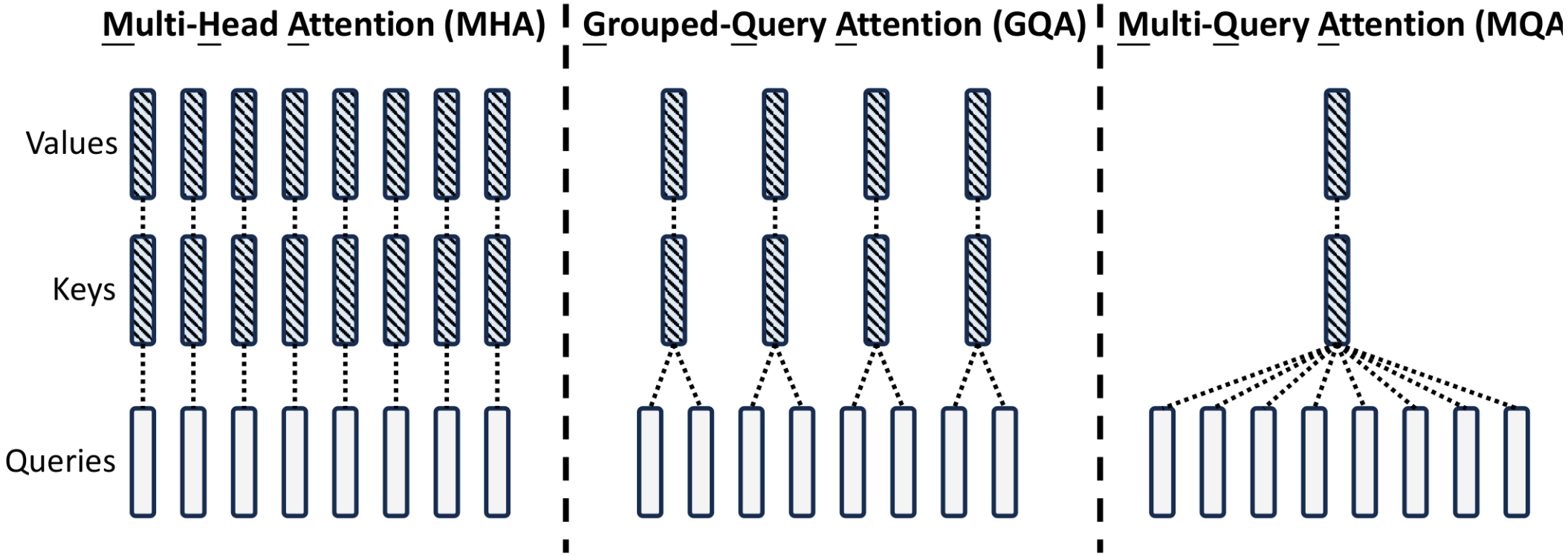
KV Cache = Key–Value Cache；每层保存历史键、值向量

当前处理 C	Key（键）	Value（值）
历史 A、B	复用已保存的 k_A、k_B	复用已保存的 v_A、v_B
当前 C	计算 k_C 并加入缓存	计算 v_C 并加入缓存
本步输出	q_C 与这些 Key 匹配	按匹配权重汇总这些 Value

缓存的是中间向量，不是原文或答案；因果注意力允许复用

只需当前 Query；历史 Key、Value 会被后续查询反复使用

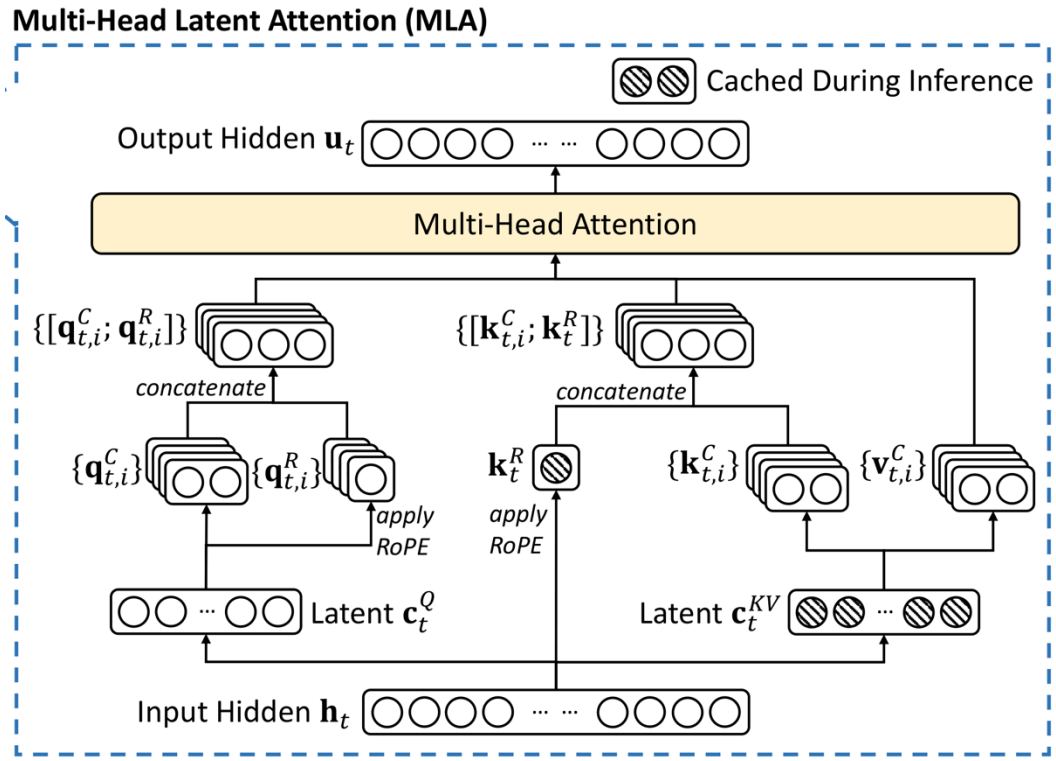
MHA、GQA 与 MQA



Grouped-Query Attention（GQA）：每组查询头共用同一组 K/V

Multi-Query Attention（MQA）：所有查询头共用同一组 K/V

Multi-head Latent Attention（MLA）



每个历史位置保存：低维内容向量 + 位置 Key

当前 Query 读取这些缓存；各头计算后合并输出

Low-Rank Key-Value Joint Compression

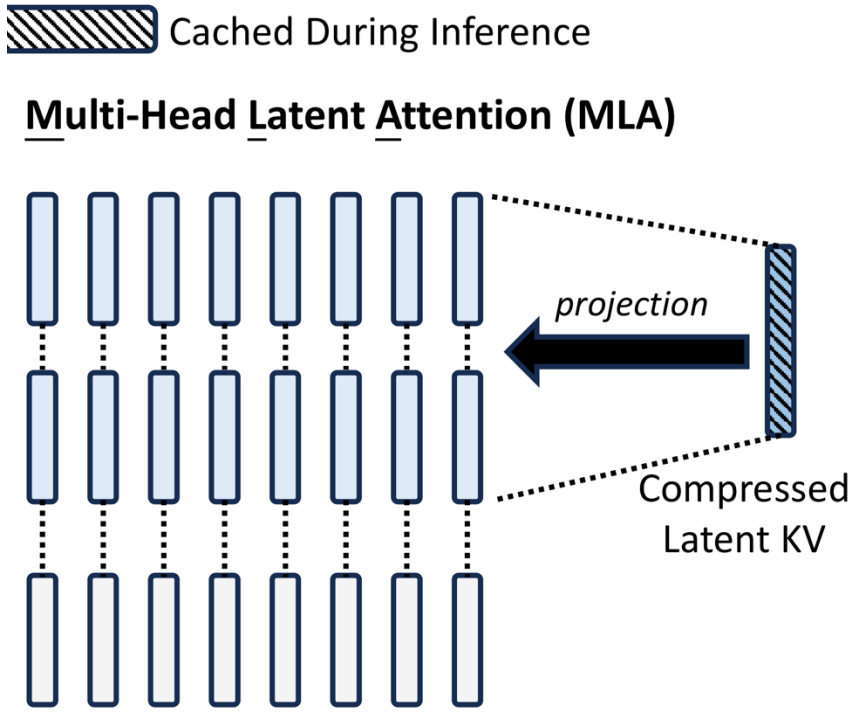
$$\mathbf{c}_t^{KV} = W^{DKV}\mathbf{h}_t,$$

$$\mathbf{k}_t^C = W^{UK}\mathbf{c}_t^{KV},$$

$$\mathbf{v}_t^C = W^{UV}\mathbf{c}_t^{KV},$$

h_t 为当前输入；降维得到分量更少的 c_t^{KV}，保存它作为缓存
升维：与不同矩阵相乘，生成分量更多的内容 K、V

MLA：Inference 中的投影合并



先变换当前 Query，再与低维缓存匹配；不逐位置生成高维 K
先加权汇总低维缓存，再统一升维；不逐位置生成高维 V

Decoupled Rotary Position Embedding

$$
\begin{aligned}
[\mathbf{q}_{t,1}^R; \mathbf{q}_{t,2}^R; \ldots; \mathbf{q}_{t,n_h}^R] = \mathbf{q}_t^R &= \mathrm{RoPE}(W^{QR}\mathbf{c}_t^Q), \\
\mathbf{k}_t^R &= \mathrm{RoPE}(W^{KR}\mathbf{h}_t), \\
\mathbf{q}_{t,i} &= [\mathbf{q}_{t,i}^C; \mathbf{q}_{t,i}^R], \\
\mathbf{k}_{t,i} &= [\mathbf{k}_{t,i}^C; \mathbf{k}_t^R],
\end{aligned}
$$

C 为内容分量，R 为旋转位置分量；[;] 表示拼接

内容路径保留投影合并；位置 Key 单独缓存、供各头共享

MLA：Attention Output

$$\mathbf{o}_{t,i} = \sum_{j=1}^{t} \mathrm{Softmax}_j\left(\frac{\mathbf{q}_{t,i}^{T}\mathbf{k}_{j,i}}{\sqrt{d_h + d_h^R}}\right)\mathbf{v}_{j,i}^{C},$$

$$\mathbf{u}_t = W^O\left[\mathbf{o}_{t,1}; \mathbf{o}_{t,2}; \ldots; \mathbf{o}_{t,n_h}\right],$$

t 为当前位置，j 遍历历史，i 为头编号；点积含内容与位置
Softmax 得到各位置权重，汇总内容 Value，再合并各头

MLA：KV Cache Comparison

Attention Mechanism	KV Cache per Token (# Element)
Multi-Head Attention (MHA)	$2n_h d_h l$
Grouped-Query Attention (GQA)	$2n_g d_h l$
Multi-Query Attention (MQA)	$2d_h l$
MLA (Ours)	$(d_c + d_h^R)l \approx \frac{9}{2} d_h l$

每 token、所有层：MHA 按头存 K、V；MLA 存 latent 与位置 K
总字节数还需乘：序列长度 × 批内序列数 × 每元素字节数

MLA：架构与实现的边界

低秩表示：减少向量分量数，训练中学习保留有用信息

实际缓存：每个位置的低维内容向量与位置 Key

推理计算：重排矩阵乘法，不反复生成所有历史的高维 K/V

访问范围：每个新查询仍可能读取全部历史

本节内容

CONTENTS

Sparse Attention 与硬件执行

MLA 缩小每个位置，但查询仍要读取长历史

Native Sparse Attention（NSA）：少读历史

FlashAttention：少搬运分数与权重矩阵

从减少历史读取，到减少显存读写

Sparse Attention：访问集合与输出

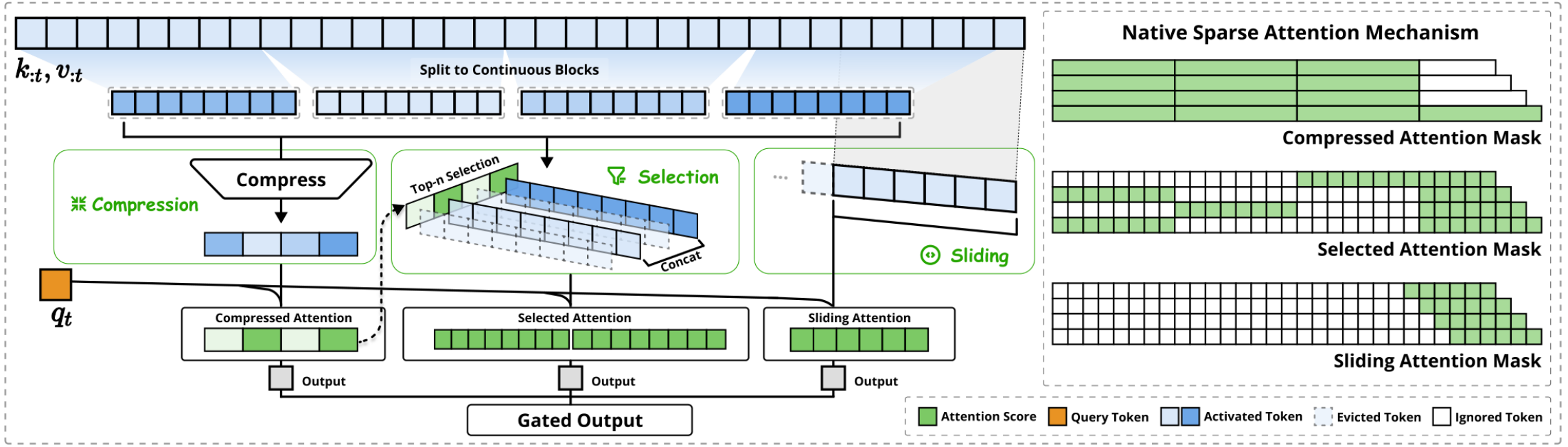
给定指数化分数 a=[1,2,1,4]，标量值 v=[2,4,6,8]

访问方式	访问位置	归一化权重	输出
全注意力	A、B、C、D	[1, 2, 1, 4] / 8	48 / 8 = 6
稀疏示例	只取 B、D	[2, 4] / 6	40 / 6 = 20/3

少读位置后，Softmax 分母与加权结果通常都会改变

此例手动选择 B、D；实际方法还必须解决怎样选择

NSA：从全局概况到局部细节



历史键值 → 压缩块、选中块、近期窗口 → 各自注意力输出

压缩得分帮助选块；门控合并三路信息，不是三个独立算法

NSA：连续块的压缩表示

$$\tilde{K}_t^{\text{cmp}} = f_K^{\text{cmp}}(\mathbf{k}_{:t}) = \left\{ \varphi(\mathbf{k}_{id+1:id+l}) \middle| 0 \leqslant i \leqslant \left\lfloor \frac{t-l}{d} \right\rfloor \right\}$$

块长 l=2、步长 d=2：连续块为 (a,b)、(c,d)、(e,f)

φ 是可学习聚合：每块形成一个键和值，不是生成文字摘要

NSA：块得分与细节读取

$$\mathbf{p}_t^{\text{cmp}} = \text{Softmax}\left(\mathbf{q}_t^T \tilde{K}_t^{\text{cmp}}\right),$$

课堂特例：压缩块与选择块相同，当前只选得分最高的一块

块	包含位置	指数化分数	归一化得分
A	a、b	1	1/5 = 0.2
B	c、d	3	3/5 = 0.6
C	e、f	1	1/5 = 0.2

选中 B 块后，读取 c、d 各自的键和值，再计算注意力
本次没有选 A、C，不代表把它们从历史缓存永久删除

NSA：窗口补充近期信息

窗口规则：始终读取当前位置和前两个位置（课堂示例）

输入位置 / 查询	窗口内：读取各位置 K/V	窗口外：本分支不读
f / q_f	d、e、f	a、b、c
g / q_g	e、f、g	a、b、c、d

单设窗口处理近邻关系，减少局部模式对压缩、选择分支的干扰

Query 匹配窗口内 Key → Softmax → 加权汇总 Value

NSA：三路输出的门控合并

$$\mathbf{o}_t^* = \sum_{c \in C} g_t^c \cdot \mathrm{Attn}(\mathbf{q}_t, \tilde{K}_t^c, \tilde{V}_t^c).$$

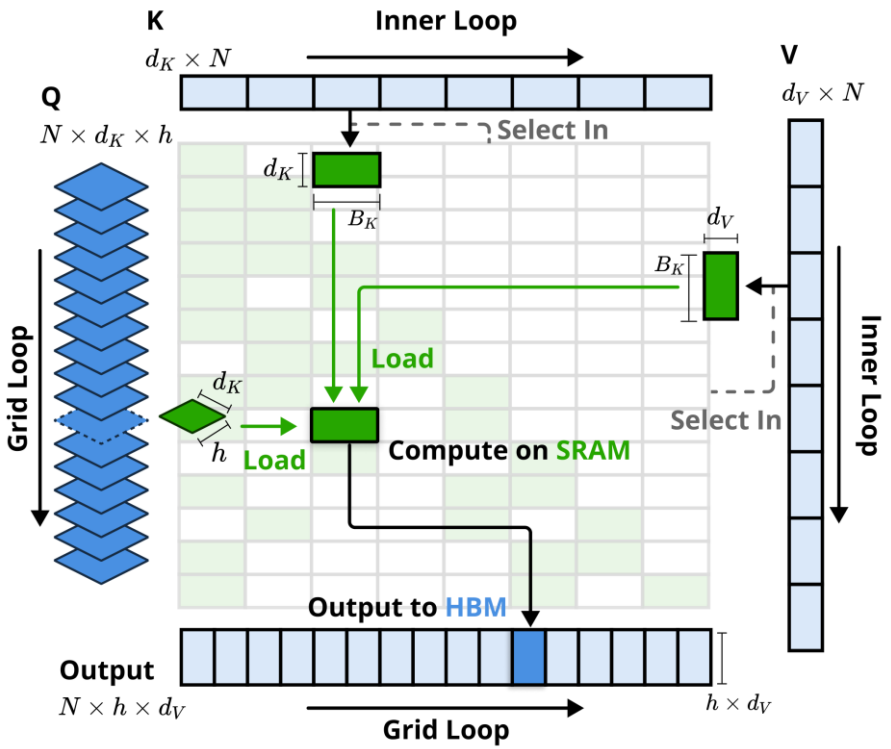
课堂代入：输出 o 与门系数 g 均为示例值

分支	输出 o	门 g	贡献 g × o
Compression	2	0.5	1
Selection	6	0.25	1.5
Window	4	0.5	2

本例总输出为 1 + 1.5 + 2 = 4.5

sigmoid 使每个 g 介于 0 与 1；各分支系数之和不必为 1

稀疏访问与 GPU 数据搬运



HBM（High Bandwidth Memory）：计算芯片外的大容量显存

SRAM（Static Random Access Memory）：芯片内部高速存储

FlashAttention：分块计算与显存读写

“片上”：GPU 芯片内部的高速存储（共享内存、寄存器）

计算对象	普通分步实现	FlashAttention
分数 QK^T	完整矩阵写入显存	小块读入芯片后打分
Softmax 权重	读分数、写权重矩阵	芯片内累加归一化分母
加权输出	读权重与 V，再相乘	芯片内累加，输出写回显存

每次读入一小块，在 GPU 内计算并累加；最后把输出写回显存

省去完整分数、权重矩阵的显存读写（Input/Output，IO）

FlashAttention：跨块归一化算例

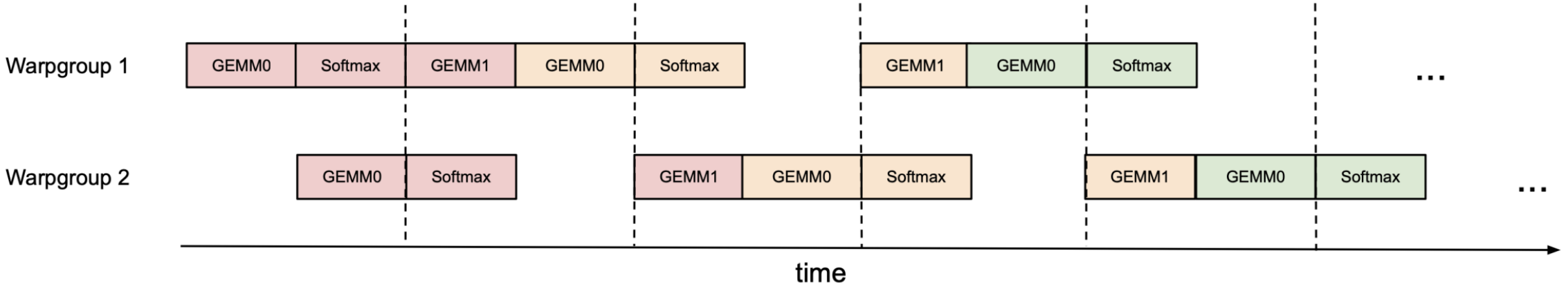
沿用 a=[1,2,1,4]、v=[2,4,6,8]；这里不删除任何位置

处理范围	指数和 Z	加权值和 U	U / Z
A、B 块	1+2 = 3	1×2+2×4 = 10	10/3
C、D 块	1+4 = 5	1×6+4×8 = 38	38/5
两块合并	3+5 = 8	10+38 = 48	48/8 = 6

正确做法：先合并分子与分母，再除一次；不是相加两块均值

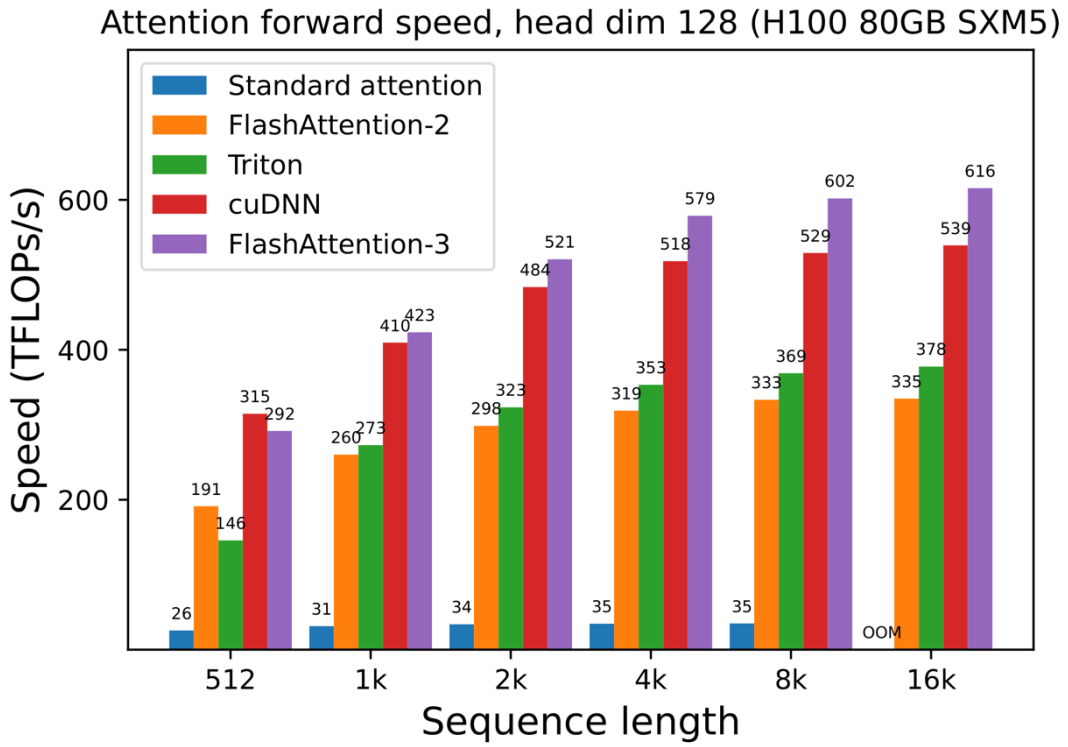
为避免指数溢出，按最大分数调整尺度；分子、分母同乘一个系数

FlashAttention-3：Pingpong Scheduling



GEMM（General Matrix Multiplication）：通用矩阵乘法
一组线程做 Softmax，另一组做矩阵乘法，重叠执行以减少等待

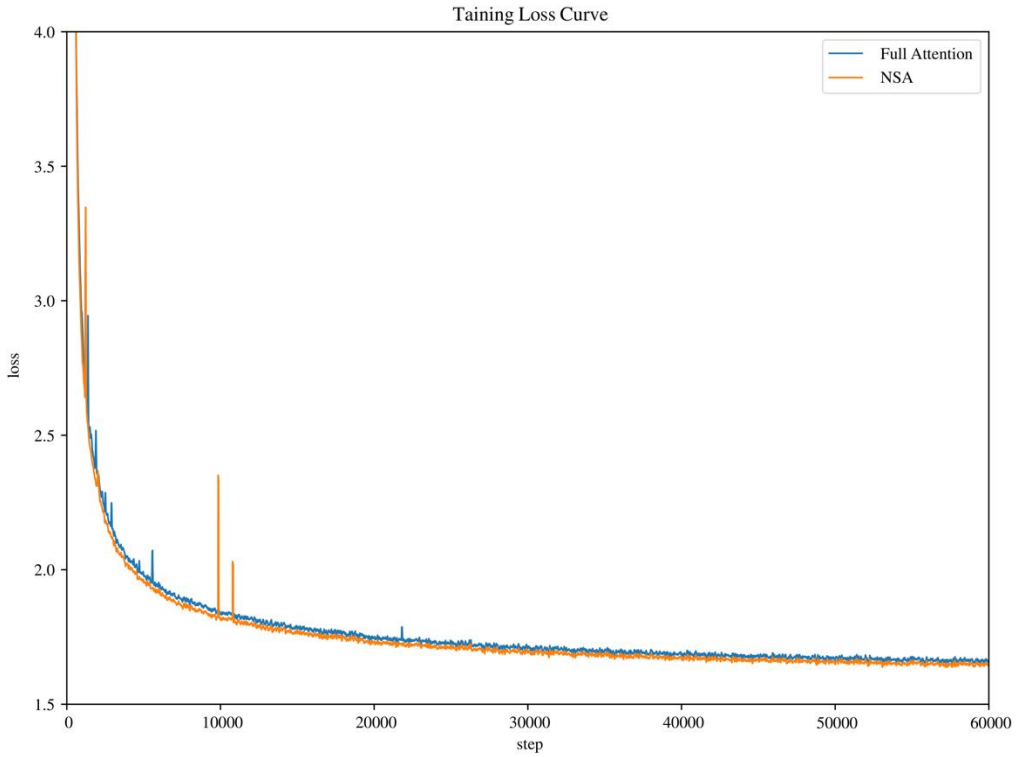
FlashAttention-3：H100 Benchmark



H100 80GB；FP16（16 位浮点）；Causal；Head dim 128

16k：FlashAttention-3 / -2 为 616 / 335 万亿次浮点运算/秒

NSA：Pretraining Loss



横轴：训练更新次数；纵轴：下一 token 预测损失（越低越好）
蓝线：Full Attention；橙线：NSA；比较学习过程是否稳定

Sparse Attention 与 FlashAttention

比较维度	NSA	FlashAttention
参与的信息	压缩、选择、局部窗口	给定 Mask 内的注意力
核心变化	表示与访问结构	减少显存读写，安排执行顺序
效果边界	需要相应训练与评估	浮点及量化误差需检查

本节内容

CONTENTS

Gated DeltaNet 与 Hybrid Attention

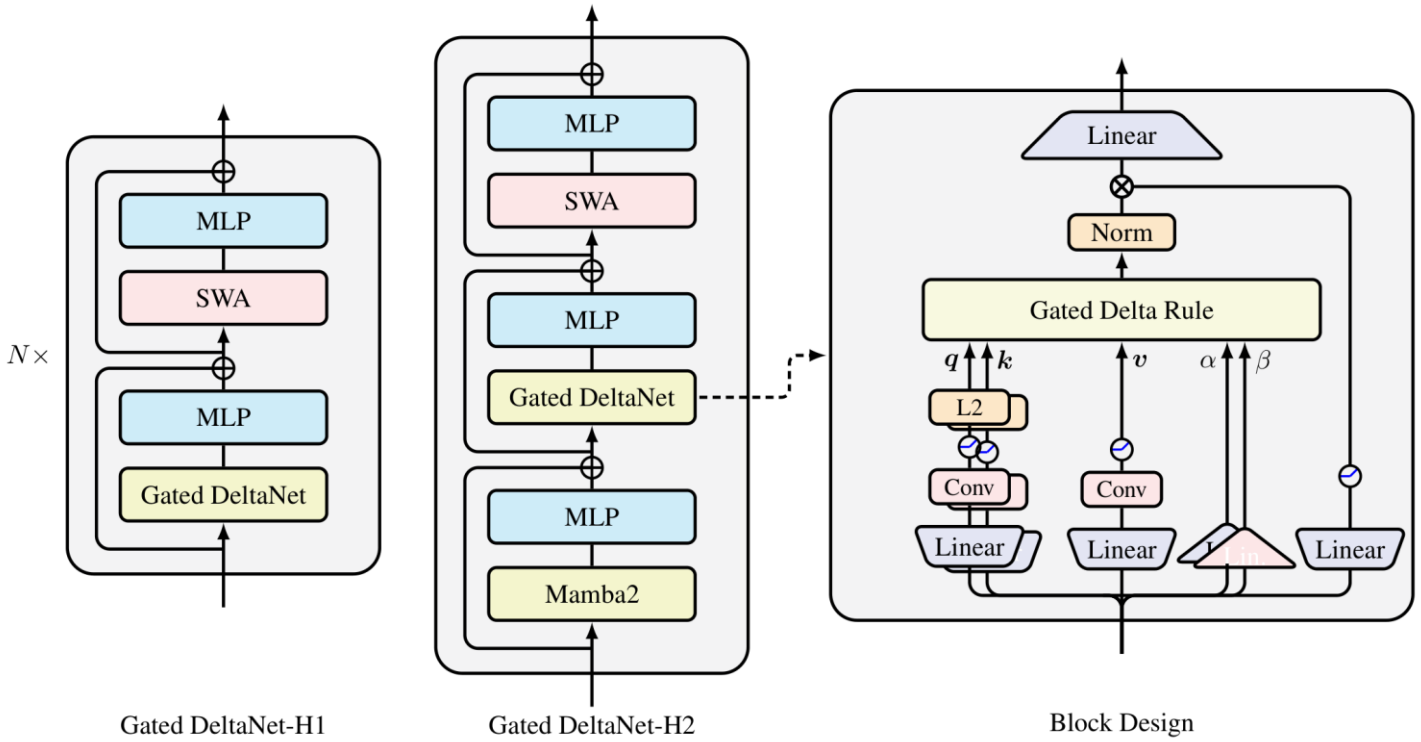
目标：利用长前文，减少逐位置历史存储

Gated DeltaNet：维护可遗忘、可纠正的状态

Hybrid Attention：结合状态记忆与逐位置读取

历史存储的代价 → 有限状态 → 遗忘与纠正 → 混合读取

Gated DeltaNet：完整架构与模块定位



H1：Gated DeltaNet 与 SWA 混合；SWA 只读窗口内信息

SWA = Sliding Window Attention；MLP = Multilayer Perceptron

线性注意力：状态递推的由来

目的：用固定形状的 S 汇总历史，不逐条保存历史 K/V

无 Softmax：Σ_i v_i(k_i^T q_t) = (Σ_i v_i k_i^T)q_t = S_t q_t

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \boldsymbol{v}_t \boldsymbol{k}_t^{\intercal} \in \mathbb{R}^{d_v \times d_k}, \qquad \boldsymbol{o}_t = \mathbf{S}_t \boldsymbol{q}_t \in \mathbb{R}^{d_v}$$

S_0=[0,0]；S_{t-1} 为旧状态，v_t k_t^T 为本次写入

当前键和值	外积 v k^T	更新后的 S
k_A=[1,0]^T，v_A=2	[2,0]	[0,0]+[2,0]=[2,0]
k_B=[0,1]^T，v_B=5	[0,5]	[2,0]+[0,5]=[2,5]
再次写 A，v_A=7	[7,0]	[2,5]+[7,0]=[9,5]

查询 q=k_A=[1,0]^T：读取 [9,5]q = 9 = 2+7

累加保留两次贡献；若任务要求 A=7，就需要差值纠正

Delta Rule：已有内容的定向纠正

$$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^T) + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^T$$

S_old=[2,5]，k=[1,0]ᵀ，目标 v=7，β=1

步骤	代入计算
读旧关联	S_old k = 2
算修正量	v − S_old k = 7 − 2 = 5
写回状态	[2,5] + 5×[1,0] = [7,5]

S_new = S_old + β(v − S_old k)kᵀ：已有状态加上修正量

差值更新：7−2=5，写回得到 [7,5]，查询 A 读到 7

Gated Delta Rule：旧状态保留与纠正

旧背景可能干扰当前读取；但更新 A 不代表 B 已过时

$$\mathbf{S}_t = \mathbf{S}_{t-1}\left(\alpha_t(\mathbf{I} - \beta_t \boldsymbol{k}_t \boldsymbol{k}_t^\intercal)\right) + \beta_t \boldsymbol{v}_t \boldsymbol{k}_t^\intercal$$

S_old=[2,5]，k=[1,0]ᵀ，v=7，β=1

任务条件	α	新状态与含义
只更新 A；B 仍有效	1	[7,5]：保留 B
A 更新；旧背景需减弱	0.5	[7,2.5]：减弱 B
衰减分支的计算	0.5	[1,2.5]+[6,0]=[7,2.5]

α：旧状态保留多少；β：当前关联修正多少

α 接近 1 时尽量保留；过度衰减也会丢失有用信息

Hybrid Attention：有限状态与逐位置读取

固定状态把历史关联合在一起，不能保证保留所有细节

比较维度	Gated DeltaNet	注意力层
历史怎样保存	固定形状的状态 S	各位置的 K/V
当前怎样读取	查询乘状态：S q	逐位置匹配并加权
主要代价	关联可能相互干扰	保存与读取历史 K/V

Hybrid 指层的组合；SWA 仅指滑动窗口注意力

窗口层只读取近期；全局层仍保留随历史增长的缓存

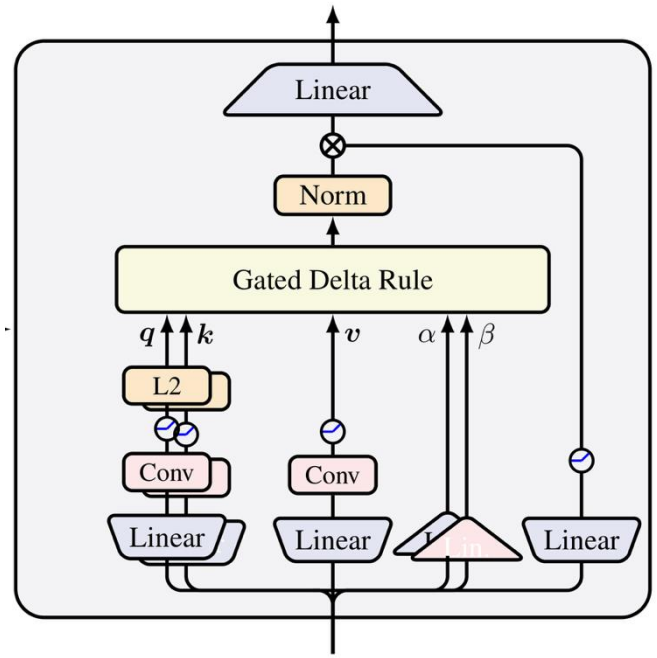
Gated DeltaNet：完整模块的数据流

写入：k 与 v

k 定方向；v 是内容

读取：q

q 决定查询什么



Block Design

当前输出 o

处理后送往后续层

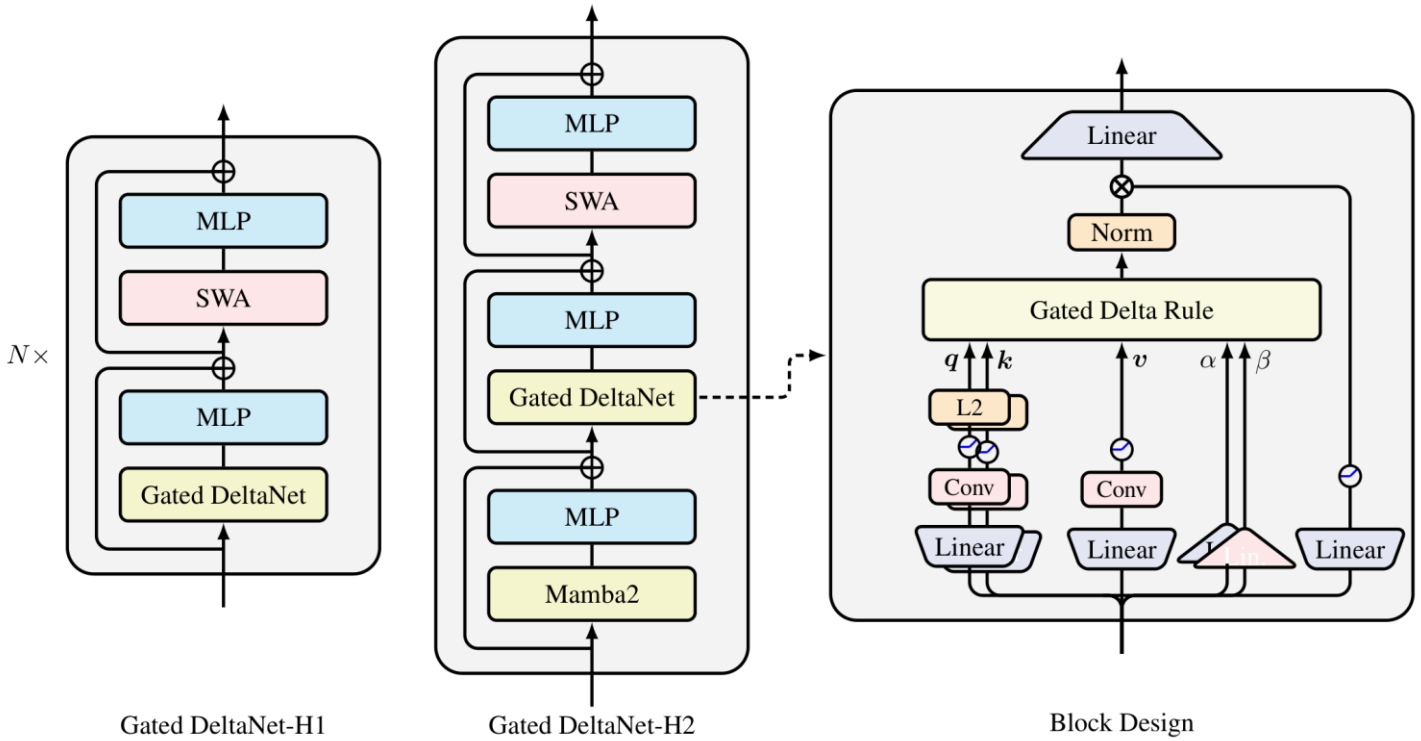
状态控制 α/β

旧状态衰减与纠正

当前特征 → 生成 q/k/v 与 α/β → 更新状态 → 查询读取

S 留给后续位置；o 经归一化、输出门与投影，送往上层

Hybrid Attention：层的串联与分工



H1：Gated DeltaNet → MLP → SWA → MLP

Hybrid 是层的组合；SWA 只是其中的窗口注意力

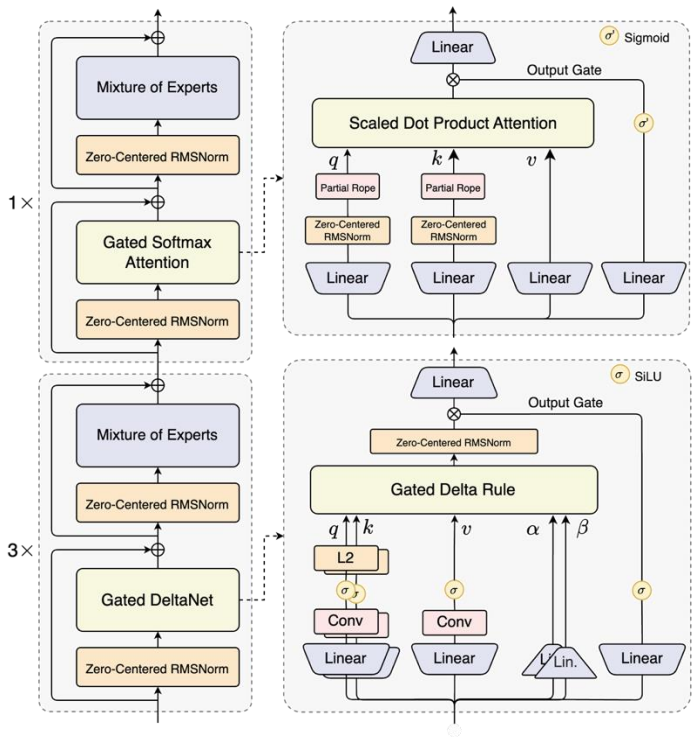
Qwen3-Next：两类层的完整架构

上层：全局注意力

逐位置读取历史 K/V

下层：递归模块

更新固定形状状态 S



右上：注意力内部

匹配 → 汇总 → 输出门

右下：递归模块内部

衰减 → 纠正 → 读出

这里的 Hybrid：Gated DeltaNet 与全局注意力，而非 SWA

Mixture of Experts（MoE）：路由选专家，逐位置变换特征

Qwen3-Next：Hybrid Layout

层间特征依次传递；两类模块不是并行计算后直接相加

重复单元	位置间信息交换	后接子层
连续 3 层	Gated DeltaNet	Mixture of Experts（MoE）
随后 1 层	Gated Attention	Mixture of Experts（MoE）
单元重复 12 次	共 48 层	保留 Attention 的 KV Cache

三层递归 + 一层全局注意力：减少使用全局缓存的层数

仍有全局注意力层：整个模型的历史存储并非固定

Gated DeltaNet 与 Hybrid：机制与用途

目标：减少历史处理开销，同时保留对上下文的利用能力

机制	具体操作	用途
状态 S	汇总历史，再以查询读取	固定形状，不逐条缓存
Delta Rule	读已有值，按差值纠正	定向更新当前关联
门控 α/β	旧状态衰减、控制修正量	调节遗忘与写入
Hybrid Attention	串联递归层与注意力层	补充逐位置访问

状态留给后续位置；读出的特征经处理后交给后续层

固定状态不是无损历史；混合结构也不是取消全部缓存

本节内容

CONTENTS

Self-Attention 与 Transformer

Multi-head Latent Attention（MLA）

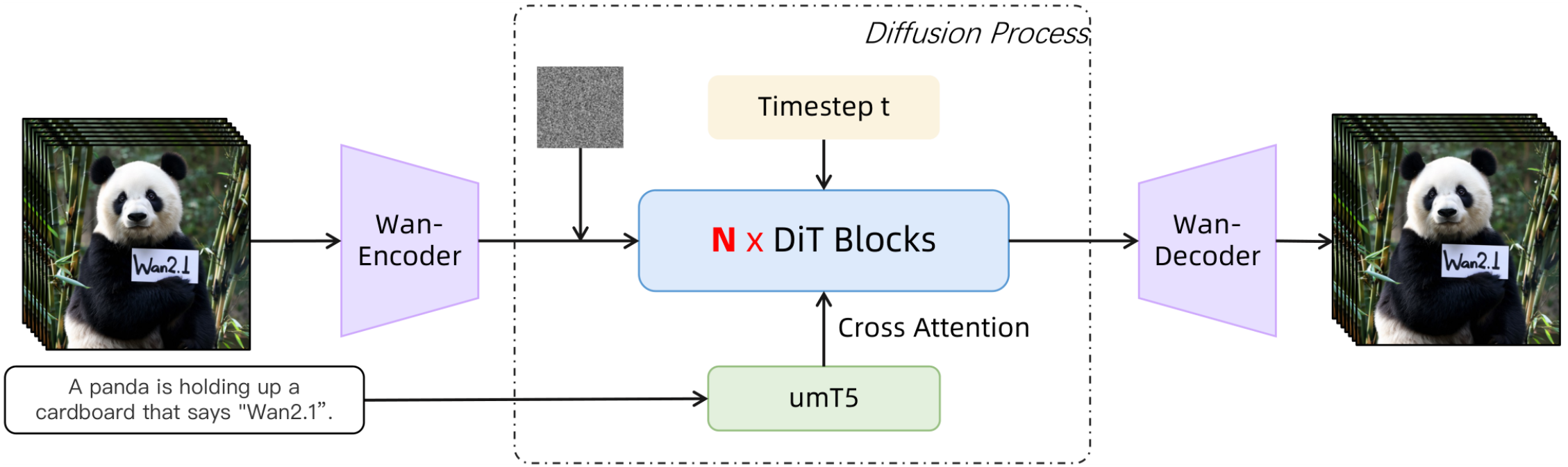
Sparse Attention 与硬件执行

Gated DeltaNet 与 Hybrid Attention

Video Diffusion Transformer

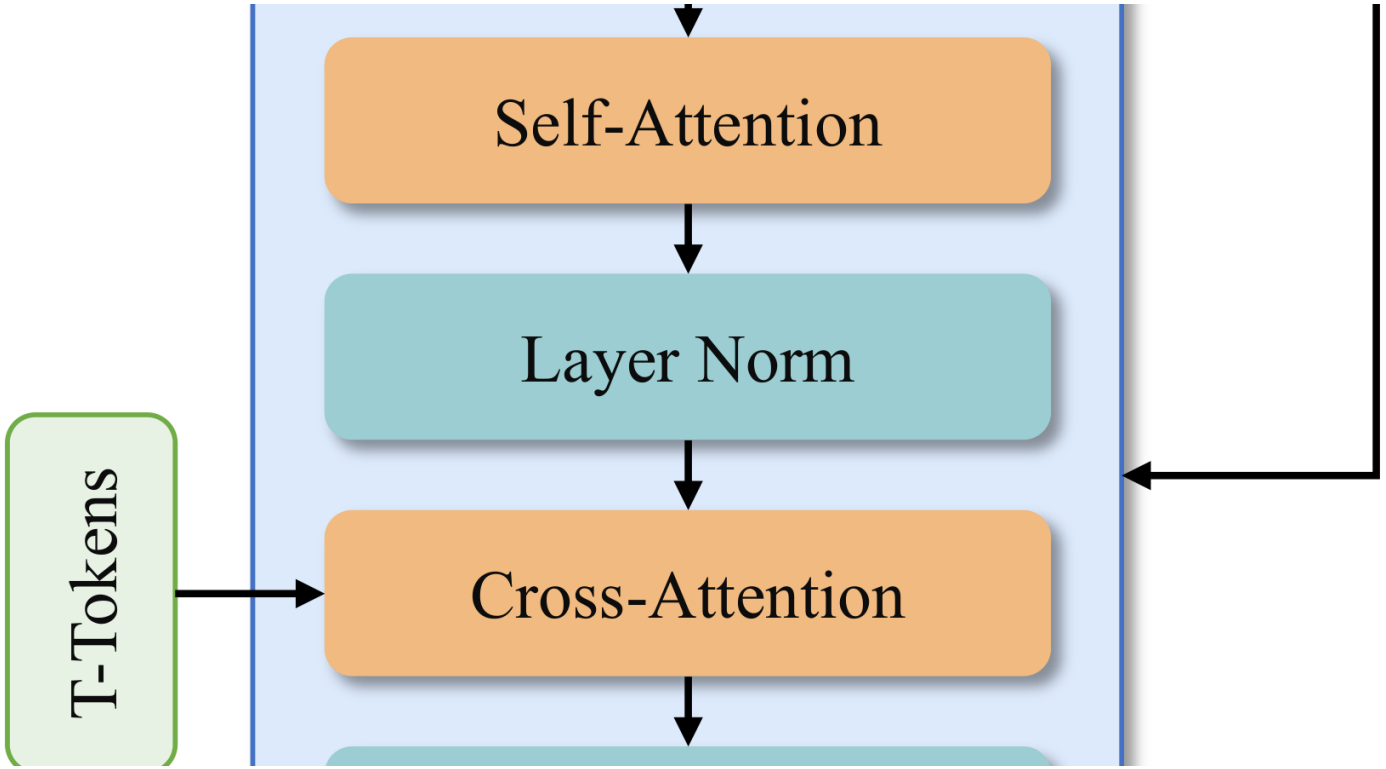
从文本位置到视频的时空 token

Wan：Video Generation Architecture



Diffusion Transformer（DiT）：扩散 Transformer
在视频 latent（压缩表示）上，从噪声按文本条件逐步生成

Wan：Transformer Block



Self-Attention：Query、Key、Value 都来自视频 token

Cross-Attention：视频 Query 匹配文本 Key，再汇总文本 Value

Wan：Spatio-temporal RoPE

```
seq_len = f * h * w
freqs_i = torch.cat([
    freqs[0][:f].view(f, 1, 1, -1).expand(f, h, w, -1),
    freqs[1][:h].view(1, h, 1, -1).expand(f, h, w, -1),
    freqs[2][:w].view(1, 1, w, -1).expand(f, h, w, -1)
], dim=-1).reshape(seq_len, 1, -1)
```

f、h、w：时间、高度、宽度方向的位置数；freqs：旋转因子
cat 拼接特征；expand 让其他网格位置共用同一方向的因子

文本生成与视频生成

问题	自回归文本示例	Wan 视频示例
Token	文字序列表示	压缩视频的时空小块
位置	序列位置	时间、高度、宽度
条件连接	提示位于序列前缀	文本 Cross-Attention

总结：注意力架构的变化

方法	主要改变	仍需检查
MLA	历史表示的维度与缓存	位置分量与推理实现
NSA	访问范围与信息粒度	训练效果与选择开销
FlashAttention	显存读写与执行顺序	硬件、精度与形状
Gated DeltaNet	历史状态的更新方式	容量、检索与混合层

课后思考题

MLA 为什么还要缓存位置 Key?

NSA 的 Compression 与 MLA 的压缩有何不同?

Hybrid 模型为什么仍可能需要增长的 KV Cache?

视频帧位置与生成 timestep 为什么不能混用?